

Vector dokumentacija

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T > Class Template Reference	5
3.1.1 Member Typedef Documentation	7
3.1.1.1 allocator_type	7
3.1.1.2 const_iterator	7
3.1.1.3 const_pointer	7
3.1.1.4 const_reference	7
3.1.1.5 const_reverse_iterator	7
3.1.1.6 difference_type	7
3.1.1.7 iterator	7
3.1.1.8 pointer	7
3.1.1.9 reference	8
3.1.1.10 reverse_iterator	8
3.1.1.11 size_type	8
3.1.1.12 value_type	8
3.1.2 Constructor & Destructor Documentation	8
3.1.2.1 Vector() [1/6]	8
3.1.2.2 Vector() [2/6]	8
3.1.2.3 Vector() [3/6]	8
3.1.2.4 Vector() [4/6]	8
3.1.2.5 Vector() [5/6]	9
3.1.2.6 Vector() [6/6]	9
3.1.2.7 ~Vector()	9
3.1.3 Member Function Documentation	9
3.1.3.1 assign() [1/3]	9
3.1.3.2 assign() [2/3]	9
3.1.3.3 assign() [3/3]	9
3.1.3.4 at() [1/2]	9
3.1.3.5 at() [2/2]	10
3.1.3.6 back() [1/2]	10
3.1.3.7 back() [2/2]	10
3.1.3.8 begin() [1/2]	10
3.1.3.9 begin() [2/2]	10
3.1.3.10 capacity()	10
3.1.3.11 cbegin()	10
3.1.3.12 cend()	10
3.1.3.13 clear()	10

3.1.3.14	crbegin()	11
3.1.3.15	crend()	11
3.1.3.16	data() [1/2]	11
3.1.3.17	data() [2/2]	11
3.1.3.18	emplace()	11
3.1.3.19	emplace_back()	11
3.1.3.20	empty()	11
3.1.3.21	end() [1/2]	11
3.1.3.22	end() [2/2]	12
3.1.3.23	erase() [1/2]	12
3.1.3.24	erase() [2/2]	12
3.1.3.25	front() [1/2]	12
3.1.3.26	front() [2/2]	12
3.1.3.27	get_allocator()	12
3.1.3.28	insert() [1/5]	12
3.1.3.29	insert() [2/5]	12
3.1.3.30	insert() [3/5]	13
3.1.3.31	insert() [4/5]	13
3.1.3.32	insert() [5/5]	13
3.1.3.33	max_size()	13
3.1.3.34	operator=() [1/3]	13
3.1.3.35	operator=() [2/3]	13
3.1.3.36	operator=() [3/3]	13
3.1.3.37	operator[]() [1/2]	14
3.1.3.38	operator[]() [2/2]	14
3.1.3.39	pop_back()	14
3.1.3.40	push_back() [1/2]	14
3.1.3.41	push_back() [2/2]	14
3.1.3.42	rbegin() [1/2]	14
3.1.3.43	rbegin() [2/2]	14
3.1.3.44	rend() [1/2]	14
3.1.3.45	rend() [2/2]	15
3.1.3.46	reserve()	15
3.1.3.47	resize() [1/2]	15
3.1.3.48	resize() [2/2]	15
3.1.3.49	shrink_to_fit()	15
3.1.3.50	size()	15
3.1.3.51	swap()	15
4	File Documentation	17
4.1	Vector.h File Reference	17
4.1.1	Function Documentation	18

4.1.1.1 operator!=(())	18
4.1.1.2 operator<()	18
4.1.1.3 operator<=()	18
4.1.1.4 operator==(())	18
4.1.1.5 operator>()	18
4.1.1.6 operator>=()	18
4.1.1.7 swap()	19
4.2 Vector.h	19
Index	25

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T >	5
-----------------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Vector.h	17
------------------------------------	----

Chapter 3

Class Documentation

3.1 `Vector< T >` Class Template Reference

```
#include <Vector.h>
```

Public Types

- using `value_type` = `T`
- using `allocator_type` = `std::allocator<T>`
- using `size_type` = `std::size_t`
- using `difference_type` = `std::ptrdiff_t`
- using `reference` = `value_type&`
- using `const_reference` = `const value_type&`
- using `pointer` = `value_type*`
- using `const_pointer` = `const value_type*`
- using `iterator` = `value_type*`
- using `const_iterator` = `const value_type*`
- using `reverse_iterator` = `std::reverse_iterator<iterator>`
- using `const_reverse_iterator` = `std::reverse_iterator<const_iterator>`

Public Member Functions

- `Vector` () noexcept
- `Vector` (`size_type` count, const `T` &value=`T`())
- `Vector` (`std::initializer_list< T >` list)
- `Vector` (const `Vector` &other)
- `Vector` (`Vector` &&other) noexcept
- template<class InputIt>
 `Vector` (InputIt first, InputIt last)
- `~Vector` ()
- `Vector` & `operator=` (const `Vector` &other)
- `Vector` & `operator=` (`Vector` &&other) noexcept
- `Vector` & `operator=` (`std::initializer_list< value_type >` list)
- void `assign` (`size_type` count, const `T` &value)
- void `assign` (`std::initializer_list< T >` list)

- `template<class InputIt>`
`void assign (InputIt first, InputIt last)`
- `allocator_type get_allocator () const noexcept`
- `reference at (size_type pos)`
- `const_reference at (size_type pos) const`
- `reference operator[] (size_type pos)`
- `const_reference operator[] (size_type pos) const`
- `reference front ()`
- `const_reference front () const`
- `reference back ()`
- `const_reference back () const`
- `pointer data () noexcept`
- `const_pointer data () const noexcept`
- `iterator begin () noexcept`
- `const_iterator begin () const noexcept`
- `const_iterator cbegin () const noexcept`
- `iterator end () noexcept`
- `const_iterator end () const noexcept`
- `const_iterator cend () const noexcept`
- `reverse_iterator rbegin () noexcept`
- `const_reverse_iterator rbegin () const noexcept`
- `const_reverse_iterator crbegin () const noexcept`
- `reverse_iterator rend () noexcept`
- `const_reverse_iterator rend () const noexcept`
- `const_reverse_iterator crend () const noexcept`
- `bool empty () const noexcept`
- `size_type size () const noexcept`
- `size_type max_size () const noexcept`
- `void reserve (size_type new_cap)`
- `size_type capacity () const noexcept`
- `void shrink_to_fit ()`
- `void clear () noexcept`
- `iterator insert (const_iterator pos, const T &value)`
- `iterator insert (const_iterator pos, T &&value)`
- `iterator insert (const_iterator pos, size_type count, const T &value)`
- `template<class InputIt>`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
- `iterator insert (const_iterator pos, std::initializer_list< T > list)`
- `template<class... Args>`
`iterator emplace (const_iterator pos, Args &&... args)`
- `iterator erase (const_iterator pos)`
- `iterator erase (const_iterator first, const_iterator last)`
- `void push_back (const T &value)`
- `void push_back (T &&value)`
- `template<class... Args>`
`reference emplace_back (Args &&... args)`
- `void pop_back ()`
- `void resize (size_type count)`
- `void resize (size_type count, const value_type &value)`
- `void swap (Vector &other) noexcept`

3.1.1 Member Typedef Documentation

3.1.1.1 allocator_type

```
template<typename T>  
using Vector< T >::allocator_type = std::allocator<T>
```

3.1.1.2 const_iterator

```
template<typename T>  
using Vector< T >::const_iterator = const value_type*
```

3.1.1.3 const_pointer

```
template<typename T>  
using Vector< T >::const_pointer = const value_type*
```

3.1.1.4 const_reference

```
template<typename T>  
using Vector< T >::const_reference = const value_type&
```

3.1.1.5 const_reverse_iterator

```
template<typename T>  
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

3.1.1.6 difference_type

```
template<typename T>  
using Vector< T >::difference_type = std::ptrdiff_t
```

3.1.1.7 iterator

```
template<typename T>  
using Vector< T >::iterator = value_type*
```

3.1.1.8 pointer

```
template<typename T>  
using Vector< T >::pointer = value_type*
```

3.1.1.9 reference

```
template<typename T>
using Vector< T >::reference = value_type&
```

3.1.1.10 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

3.1.1.11 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

3.1.1.12 value_type

```
template<typename T>
using Vector< T >::value_type = T
```

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Vector() [1/6]

```
template<typename T>
Vector< T >::Vector () [inline], [noexcept]
```

3.1.2.2 Vector() [2/6]

```
template<typename T>
Vector< T >::Vector (
    size_type count,
    const T & value = T()) [inline], [explicit]
```

3.1.2.3 Vector() [3/6]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > list) [inline]
```

3.1.2.4 Vector() [4/6]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

3.1.2.5 Vector() [5/6]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

3.1.2.6 Vector() [6/6]

```
template<typename T>
template<class InputIt>
Vector< T >::Vector (
    InputIt first,
    InputIt last) [inline]
```

3.1.2.7 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

3.1.3 Member Function Documentation

3.1.3.1 assign() [1/3]

```
template<typename T>
template<class InputIt>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

3.1.3.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value) [inline]
```

3.1.3.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > list) [inline]
```

3.1.3.4 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
    size_type pos) [inline]
```

3.1.3.5 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
    size_type pos) const [inline]
```

3.1.3.6 back() [1/2]

```
template<typename T>
reference Vector< T >::back () [inline]
```

3.1.3.7 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const [inline]
```

3.1.3.8 begin() [1/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline], [noexcept]
```

3.1.3.9 begin() [2/2]

```
template<typename T>
iterator Vector< T >::begin () [inline], [noexcept]
```

3.1.3.10 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const [inline], [noexcept]
```

3.1.3.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline], [noexcept]
```

3.1.3.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline], [noexcept]
```

3.1.3.13 clear()

```
template<typename T>
void Vector< T >::clear () [inline], [noexcept]
```


3.1.3.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline], [noexcept]
```

3.1.3.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline], [noexcept]
```

3.1.3.16 data() [1/2]

```
template<typename T>
const_pointer Vector< T >::data () const [inline], [noexcept]
```

3.1.3.17 data() [2/2]

```
template<typename T>
pointer Vector< T >::data () [inline], [noexcept]
```

3.1.3.18 emplace()

```
template<typename T>
template<class... Args>
iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

3.1.3.19 emplace_back()

```
template<typename T>
template<class... Args>
reference Vector< T >::emplace_back (
    Args &&... args) [inline]
```

3.1.3.20 empty()

```
template<typename T>
bool Vector< T >::empty () const [inline], [noexcept]
```

3.1.3.21 end() [1/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline], [noexcept]
```

3.1.3.22 end() [2/2]

```
template<typename T>
iterator Vector< T >::end () [inline], [noexcept]
```

3.1.3.23 erase() [1/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

3.1.3.24 erase() [2/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

3.1.3.25 front() [1/2]

```
template<typename T>
reference Vector< T >::front () [inline]
```

3.1.3.26 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const [inline]
```

3.1.3.27 get_allocator()

```
template<typename T>
allocator_type Vector< T >::get_allocator () const [inline], [noexcept]
```

3.1.3.28 insert() [1/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value) [inline]
```

3.1.3.29 insert() [2/5]

```
template<typename T>
template<class InputIt>
iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last) [inline]
```

3.1.3.30 insert() [3/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value) [inline]
```

3.1.3.31 insert() [4/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    std::initializer_list< T > list) [inline]
```

3.1.3.32 insert() [5/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    T && value) [inline]
```

3.1.3.33 max_size()

```
template<typename T>
size_type Vector< T >::max_size () const [inline], [noexcept]
```

3.1.3.34 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

3.1.3.35 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< value_type > list) [inline]
```

3.1.3.36 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

3.1.3.37 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
    size_type pos) [inline]
```

3.1.3.38 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
    size_type pos) const [inline]
```

3.1.3.39 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

3.1.3.40 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

3.1.3.41 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

3.1.3.42 rbegin() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline], [noexcept]
```

3.1.3.43 rbegin() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline], [noexcept]
```

3.1.3.44 rend() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline], [noexcept]
```

3.1.3.45 `rend()` [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline], [noexcept]
```

3.1.3.46 `reserve()`

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap) [inline]
```

3.1.3.47 `resize()` [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count) [inline]
```

3.1.3.48 `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const value_type & value) [inline]
```

3.1.3.49 `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

3.1.3.50 `size()`

```
template<typename T>
size_type Vector< T >::size () const [inline], [noexcept]
```

3.1.3.51 `swap()`

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

The documentation for this class was generated from the following file:

- [Vector.h](#)

Chapter 4

File Documentation

4.1 Vector.h File Reference

```
#include <memory>
#include <iterator>
#include <stdexcept>
#include <algorithm>
#include <initializer_list>
#include <utility>
```

Classes

- class [Vector< T >](#)

Functions

- template<typename T>
bool [operator==](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
bool [operator!=](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
bool [operator<](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
bool [operator<=](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
bool [operator>](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
bool [operator>=](#) (const [Vector< T >](#) &left, const [Vector< T >](#) &right)
- template<typename T>
void [swap](#) ([Vector< T >](#) &left, [Vector< T >](#) &right) noexcept

4.1.1 Function Documentation

4.1.1.1 operator"!=()

```
template<typename T>
bool operator!= (
    const Vector< T > & left,
    const Vector< T > & right)
```

4.1.1.2 operator<()

```
template<typename T>
bool operator< (
    const Vector< T > & left,
    const Vector< T > & right)
```

4.1.1.3 operator<=()

```
template<typename T>
bool operator<= (
    const Vector< T > & left,
    const Vector< T > & right)
```

4.1.1.4 operator==()

```
template<typename T>
bool operator== (
    const Vector< T > & left,
    const Vector< T > & right)
```

4.1.1.5 operator>()

```
template<typename T>
bool operator> (
    const Vector< T > & left,
    const Vector< T > & right)
```

4.1.1.6 operator>=()

```
template<typename T>
bool operator>= (
    const Vector< T > & left,
    const Vector< T > & right)
```


4.1.1.7 swap()

```
template<typename T>
void swap (
    Vector< T > & left,
    Vector< T > & right) [noexcept]
```

4.2 Vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <memory>
00005 #include <iterator>
00006 #include <stdexcept>
00007 #include <algorithm>
00008 #include <initializer_list>
00009 #include <utility>
00010
00011 template <typename T>
00012 class Vector {
00013     public:
00014         // Member types
00015         using value_type = T;
00016         using allocator_type = std::allocator<T>;
00017         using size_type = std::size_t;
00018         using difference_type = std::ptrdiff_t;
00019         using reference = value_type&;
00020         using const_reference = const value_type&;
00021         using pointer = value_type*;
00022         using const_pointer = const value_type*;
00023         using iterator = value_type*;
00024         using const_iterator = const value_type*;
00025         using reverse_iterator = std::reverse_iterator<iterator>;
00026         using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00027
00028         // Member functions
00029         Vector() noexcept : data_(nullptr), size_(0), capacity_(0) {} //Default constructor
00030
00031         explicit Vector(size_type count, const T& value = T()) //Constructor with count and default value:
00032         v(count, value)
00033             : data_(nullptr), size_(0), capacity_(0) {
00034             if(count > 0) {
00035                 data_ = allocate(count);
00036                 size_ = count;
00037                 capacity_ = count;
00038                 for(size_type i = 0; i < count; i++) {
00039                     construct_at(data_ + i, value);
00040                 }
00041             }
00042
00043         Vector(std::initializer_list<T> list) //Constructor with in itializer list: v{a, b, c}
00044             : data_(nullptr), size_(0), capacity_(0) {
00045                 data_ = allocate(list.size());
00046                 capacity_ = list.size();
00047                 for(const T& val : list) {
00048                     construct_at(data_ + size_, val);
00049                     size_++;
00050                 }
00051             }
00052
00053         Vector(const Vector& other) //Copy constructor
00054             : data_(nullptr), size_(0), capacity_(0) {
00055                 copy_from(other);
00056             }
00057
00058         Vector(Vector&& other) noexcept //Move constructor
00059             : data_(nullptr), size_(0), capacity_(0) {
00060                 move_from(std::move(other));
00061             }
00062
00063     template<class InputIt>
00064     Vector(InputIt first, InputIt last) //Constructor with range: v(x.begin(), x.end())
00065         : data_(nullptr), size_(0), capacity_(0) {
00066             for(auto it = first; it != last; it++) {
```

```

00067         push_back(*it);
00068     }
00069 }
00070 ~Vector() { //Destructor
00071     clear();
00072     if(data_) deallocate(data_, capacity_);
00073 }
00074
00075 Vector& operator=(const Vector& other) { //Copy assignment operator: x = y
00076     if(this == &other) return *this;
00077     clear();
00078     deallocate(data_, capacity_);
00079     data_ = nullptr;
00080     size_ = 0;
00081     capacity_ = 0;
00082     copy_from(other);
00083     return *this;
00084 }
00085
00086 Vector& operator=(Vector&& other) noexcept { //Move assignment operator: x = std::move(y)
00087     if(this == &other) return *this;
00088     clear();
00089     deallocate(data_, capacity_);
00090     data_ = nullptr;
00091     size_ = 0;
00092     capacity_ = 0;
00093     move_from(std::move(other));
00094     return *this;
00095 }
00096 Vector& operator=(std::initializer_list<value_type> list) { //Initializer list assignment
operator: x = {a, b, c}
00097     clear();
00098     deallocate(data_, capacity_);
00099     data_ = nullptr;
00100     size_ = 0;
00101     capacity_ = 0;
00102     data_ = allocate(list.size());
00103     capacity_ = list.size();
00104     for(const auto& val : list) {
00105         construct_at(data_ + size_, val);
00106         size_++;
00107     }
00108     return *this;
00109 }
00110 void assign(size_type count, const T& value) { //Assign new contents to the vector:
v.assign(count, value)
00111     clear();
00112     if(count > capacity_) {
00113         deallocate(data_, capacity_);
00114         data_ = allocate(count);
00115         capacity_ = count;
00116     }
00117     for(size_type i = 0; i < count; i++) {
00118         construct_at(data_ + i, value);
00119     }
00120     size_ = count;
00121 }
00122 void assign(std::initializer_list<T> list) { //Assign new contents from initializer list:
v.assign(a, b, c)
00123     assign(list.begin(), list.end());
00124 }
00125 template <class InputIt>
00126 void assign(InputIt first, InputIt last) { //Assign new contents from range: v.assign(x.begin(),
x.end())
00127     clear();
00128     for(auto it = first; it != last; it++) {
00129         push_back(*it);
00130     }
00131 }
00132
00133 allocator_type get_allocator() const noexcept {return allocator_;} //Get allocator
00134
00135 // Element access
00136 reference at(size_type pos) { //Access element with bounds checking: v.at(pos)
00137     if(pos >= size_) {
00138         throw std::out_of_range("Vector::indeksas uz ribu");
00139     }
00140     return data_[pos];
00141 }
00142
00143 const_reference at(size_type pos) const { //Access element with bounds checking: v.at(pos)
00144     if(pos >= size_) {
00145         throw std::out_of_range("Vector::at - indeksas uz ribu");
00146     }
00147     return data_[pos];
00148 }
00149

```

```

00150     reference operator[](size_type pos) {return data_[pos];} //Access element without bounds checking:
v[pos]
00151     const_reference operator[](size_type pos) const {return data_[pos];} //Access element without
bounds checking: v[pos]
00152
00153     reference front() {return data_[0];} //Access first element: v.front()
00154     const_reference front() const {return data_[0];} //Access first element: v.front()
00155
00156     reference back() {return data_[size_ - 1];} //Access last element: v.back()
00157     const_reference back() const {return data_[size_ - 1];} //Access last element: v.back()
00158
00159     pointer data() noexcept {return data_;}
00160     const_pointer data() const noexcept {return data_;}
00161
00162     // Iterators
00163     iterator begin() noexcept {return data_;} //Return iterator to beginning: v.begin()
00164     const_iterator begin() const noexcept {return data_;} //Return const iterator to beginning:
v.begin()
00165     const_iterator cbegin() const noexcept {return data_;} //Return const iterator to beginning:
v.cbegin()
00166
00167     iterator end() noexcept {return data_ + size_;} //Return iterator to end: v.end()
00168     const_iterator end() const noexcept {return data_ + size_;} //Return const iterator to end:
v.end()
00169     const_iterator cend() const noexcept {return data_ + size_;} //Return const iterator to end:
v.cend()
00170
00171     reverse_iterator rbegin() noexcept {return reverse_iterator(end());} //Return reverse iterator to
beginning: v.rbegin()
00172     const_reverse_iterator rbegin() const noexcept {return const_reverse_iterator(end());} //Return
const reverse iterator to beginning: v.rbegin()
00173     const_reverse_iterator crbegin() const noexcept {return const_reverse_iterator(end());} //Return
const reverse iterator to beginning: v.crbegin()
00174
00175     reverse_iterator rend() noexcept {return reverse_iterator(begin());} //Return reverse iterator to
end: v.rend()
00176     const_reverse_iterator rend() const noexcept {return const_reverse_iterator(begin());} //Return
const reverse iterator to end: v.rend()
00177     const_reverse_iterator crend() const noexcept {return const_reverse_iterator(begin());} //Return
const reverse iterator to end: v.crend()
00178
00179     // Capacity
00180     bool empty() const noexcept {return size_ == 0;} //Check if vector is empty: v.empty()
00181
00182     size_type size() const noexcept {return size_;} //Return number of elements: v.size()
00183
00184     size_type max_size() const noexcept { //Return maximum number of elements: v.max_size()
        return std::allocator_traits<allocator_type>::max_size(allocator_);
    }
00187
00188     void reserve(size_type new_cap) { //Request a change in capacity: v.reserve(new_cap)
        if(new_cap <= capacity_) return;
        if(new_cap > max_size()) {
            throw std::length_error("Vector::reserve - per didelis dydis");
        }
        reallocate(new_cap);
    }
00194
00195     size_type capacity() const noexcept {return capacity_;} //Return current capacity: v.capacity()
00196
00197     void shrink_to_fit() { //Reduce capacity to fit size: v.shrink_to_fit()
        if(size_ < capacity_) {
            reallocate(size_);
        }
    }
00201
00202
00203     // Modifiers
00204     void clear() noexcept { //Clear contents: v.clear()
        for(size_type i = 0; i < size_; i++) {
            destroy_at(data_ + i);
        }
        size_ = 0;
    }
00209
00210
00211     iterator insert(const_iterator pos, const T& value) { //Insert element before pos: v.insert(pos,
value)
00212         return emplace(pos, value);
00213     }
00214
00215     iterator insert(const_iterator pos, T&& value) { //Insert element before pos: v.insert(pos,
std::move(value))
00216         return emplace(pos, std::move(value));
00217     }
00218
00219     iterator insert(const_iterator pos, size_type count, const T& value) { //Insert count copies of
value before pos: v.insert(pos, count, value)
00220         if(count == 0) return const_cast<iterator>(pos);
00221         size_type index = pos - data_;

```

```

00222         if(size_ + count > capacity_) {
00223             reallocate(std::max(capacity_ * 2, size_ + count));
00224         }
00225         for(size_type i = size_; i > index; i--) {
00226             if(i < size_) {
00227                 destroy_at(data_ + i + count - 1);
00228             }
00229             construct_at(data_ + i + count - 1, std::move(data_[i - 1]));
00230         }
00231         for(size_type i = 0; i < count; i++) {
00232             construct_at(data_ + index + i, value);
00233         }
00234         size_ += count;
00235         return data_ + index;
00236     }
00237
00238     template <class InputIt>
00239     iterator insert(const_iterator pos, InputIt first, InputIt last) { //Insert elements from range
[first, last) before pos: v.insert(pos, x.begin(), x.end())
00240         size_type index = pos - data_;
00241         size_type count = std::distance(first, last);
00242         if(count == 0) return data_ + index;
00243         if(size_ + count > capacity_) {
00244             reallocate(std::max(capacity_ * 2, size_ + count));
00245         }
00246         for(size_type i = size_ + count - 1; i >= index + count; i--) {
00247             construct_at(data_ + i, std::move(data_[i - count]));
00248         }
00249         size_type i = index;
00250         for(auto it = first; it != last; it++) {
00251             construct_at(data_ + i, *it);
00252             i++;
00253         }
00254         size_ += count;
00255         return data_ + index;
00256     }
00257
00258     iterator insert(const_iterator pos, std::initializer_list<T> list) { //Insert elements from
initializer list before pos: v.insert(pos, {a, b, c})
00259         return insert(pos, list.begin(), list.end());
00260     }
00261
00262     template <class... Args>
00263     iterator emplace(const_iterator pos, Args&&... args) { //Construct element in-place before pos:
v.emplace(pos, args...)
00264         size_type index = pos - data_;
00265         grow_if_needed();
00266         for(size_type i = size_; i > index; i--) {
00267             construct_at(data_ + i, std::move(data_[i - 1]));
00268         }
00269         if(index < size_) {
00270             destroy_at(data_ + index);
00271         }
00272         construct_at(data_ + index, std::forward<Args>(args)...);
00273         size++;
00274         return data_ + index;
00275     }
00276
00277     iterator erase(const_iterator pos) { //Erase element at pos: v.erase(pos)
00278         return erase(pos, pos + 1);
00279     }
00280
00281     iterator erase(const_iterator first, const_iterator last) { //Erase elements in range [first,
last): v.erase(x.begin(), x.end())
00282         size_type first_index = first - data_;
00283         size_type last_index = last - data_;
00284         size_type count = last_index - first_index;
00285         if(count == 0) return data_ + first_index;
00286         for(size_type i = first_index; i < last_index; i++) {
00287             destroy_at(data_ + i);
00288         }
00289         for(size_type i = last_index; i < size_; ++i) {
00290             construct_at(data_ + i - count, std::move(data_[i]));
00291         }
00292         size_ -= count;
00293         return data_ + first_index;
00294     }
00295
00296     void push_back(const T& value) { //Add element to end: v.push_back(value)
00297         grow_if_needed();
00298         construct_at(data_ + size_, value);
00299         ++size_;
00300     }
00301
00302     void push_back(T&& value) { //Add element to end: v.push_back(std::move(value))
00303         grow_if_needed();
00304         construct_at(data_ + size_, std::move(value));

```

```

00305         ++size_;
00306     }
00307
00308     template <class... Args>
00309     reference emplace_back(Args&&... args) { //Construct element in-place at end:
00310         v.emplace_back(args...)
00311         grow_if_needed();
00312         construct_at(data_ + size_, std::forward<Args>(args)...);
00313         return data_[size_++];
00314     }
00315
00316     void pop_back() { //Remove last element: v.pop_back()
00317         destroy_at(data_ + --size_);
00318     }
00319
00320     void resize(size_type count) { //Change size to count: v.resize(count)
00321         resize(count, value_type{});
00322     }
00323
00324     void resize(size_type count, const value_type& value) { //Change size to count and fill new
00325         elements with value: v.resize(count, value)
00326         if(count < size_) {
00327             for(size_type i = count; i < size_; ++i)
00328                 destroy_at(data_ + i);
00329         }
00330         else if(count > size_) {
00331             if(count > capacity_) reallocate(count);
00332             for(size_type i = size_; i < count; ++i)
00333                 construct_at(data_ + i, value);
00334         }
00335         size_ = count;
00336     }
00337
00338     void swap(Vector& other) noexcept { //Swap contents with another vector: v.swap(other)
00339         std::swap(data_, other.data_);
00340         std::swap(size_, other.size_);
00341         std::swap(capacity_, other.capacity_);
00342         std::swap(allocator_, other.allocator_);
00343     }
00344
00345     private:
00346     pointer data_ = nullptr; //Pointer to the underlying array
00347     size_type size_ = 0; //Number of elements in the vector
00348     size_type capacity_ = 0; //Capacity of the vector
00349     allocator_type allocator_; //Allocator for managing memory
00350
00351     static constexpr size_type initial_capacity = 1; //Initial capacity for the vector
00352
00353     pointer allocate(size_type n) { //Allocate memory for n elements
00354         return n ? std::allocator_traits<allocator_type>::allocate(allocator_, n) : nullptr;
00355     }
00356
00357     void deallocate(pointer p, size_type n) noexcept { //Deallocate memory for n elements at pointer p
00358         if(p) std::allocator_traits<allocator_type>::deallocate(allocator_, p, n);
00359     }
00360
00361     void reallocate(size_type new_capacity) { //Reallocate memory to new capacity
00362         pointer new_data = allocate(new_capacity);
00363         for(size_type i = 0; i < size_; ++i) {
00364             construct_at(new_data + i, std::move(data_[i]));
00365             destroy_at(data_ + i);
00366         }
00367         deallocate(data_, capacity_);
00368         data_ = new_data;
00369         capacity_ = new_capacity;
00370     }
00371
00372     void grow_if_needed() { //Grow the vector if size exceeds capacity
00373         if(size_ >= capacity_) {
00374             reallocate(capacity_ == 0 ? initial_capacity : capacity_ * 2);
00375         }
00376     }
00377
00378     void construct_at(pointer p, const T& value) { //Construct element at pointer p with value
00379         std::allocator_traits<allocator_type>::construct(allocator_, p, value);
00380     }
00381
00382     void construct_at(pointer p, T&& value) {
00383         std::allocator_traits<allocator_type>::construct(allocator_, p, std::move(value));
00384     }
00385
00386     void destroy_at(pointer p) noexcept { //Destroy element at pointer p
00387         std::allocator_traits<allocator_type>::destroy(allocator_, p);
00388     }
00389
00390     void copy_from(const Vector& other) { //Copy resources from another vector
00391         if(other.size_ == 0) return;
00392         data_ = allocate(other.size_);
00393         size_ = other.size_;
00394         capacity_ = other.capacity_;

```

```

00390         for(size_type i = 0; i < size_; i++) {
00391             construct_at(data_ + i, other.data_[i]);
00392         }
00393     }
00394
00395     void move_from(Vector&& other) noexcept { //Move resources from another vector
00396         data_ = other.data_;
00397         size_ = other.size_;
00398         capacity_ = other.capacity_;
00399         allocator_ = std::move(other.allocator_);
00400         other.data_ = nullptr;
00401         other.size_ = 0;
00402         other.capacity_ = 0;
00403     }
00404 };
00405
00406 // Non-member functions
00407 template <typename T>
00408 bool operator==(const Vector<T>& left, const Vector<T>& right) { //Equality comparison: v1 == v2
00409     if(left.size() != right.size()) return false;
00410     for(typename Vector<T>::size_type i = 0; i < left.size(); i++) {
00411         if(left[i] != right[i]) return false;
00412     }
00413     return true;
00414 }
00415
00416 template <typename T>
00417 bool operator!=(const Vector<T>& left, const Vector<T>& right) { //Inequality comparison: v1 != v2
00418     return !(left == right);
00419 }
00420
00421 template <typename T>
00422 bool operator<(const Vector<T>& left, const Vector<T>& right) { //Less-than comparison: v1 < v2
00423     return std::lexicographical_compare(
00424         left.begin(), left.end(),
00425         right.begin(), right.end());
00426 }
00427
00428 template <typename T>
00429 bool operator<=(const Vector<T>& left, const Vector<T>& right) { //Less-than or equal comparison: v1
00430     <= v2
00431     return !(right < left);
00432 }
00433
00434 template <typename T>
00435 bool operator>(const Vector<T>& left, const Vector<T>& right) { //Greater-than comparison: v1 > v2
00436     return right < left;
00437 }
00438
00439 template <typename T>
00440 bool operator>=(const Vector<T>& left, const Vector<T>& right) { //Greater-than or equal comparison:
00441     v1 >= v2
00442     return !(left < right);
00443 }
00444
00445 template <typename T>
00446 void swap(Vector<T>& left, Vector<T>& right) noexcept { //Non-member swap: swap(left, right)
00447     left.swap(right);
00448 }
00449 #endif

```

Index

`~Vector`
 `Vector< T >`, 9

`allocator_type`
 `Vector< T >`, 7

`assign`
 `Vector< T >`, 9

`at`
 `Vector< T >`, 9

`back`
 `Vector< T >`, 10

`begin`
 `Vector< T >`, 10

`capacity`
 `Vector< T >`, 10

`cbegin`
 `Vector< T >`, 10

`cend`
 `Vector< T >`, 10

`clear`
 `Vector< T >`, 10

`const_iterator`
 `Vector< T >`, 7

`const_pointer`
 `Vector< T >`, 7

`const_reference`
 `Vector< T >`, 7

`const_reverse_iterator`
 `Vector< T >`, 7

`crbegin`
 `Vector< T >`, 10

`crend`
 `Vector< T >`, 11

`data`
 `Vector< T >`, 11

`difference_type`
 `Vector< T >`, 7

`emplace`
 `Vector< T >`, 11

`emplace_back`
 `Vector< T >`, 11

`empty`
 `Vector< T >`, 11

`end`
 `Vector< T >`, 11

`erase`
 `Vector< T >`, 12

`front`
 `Vector< T >`, 12

`get_allocator`
 `Vector< T >`, 12

`insert`
 `Vector< T >`, 12, 13

`iterator`
 `Vector< T >`, 7

`max_size`
 `Vector< T >`, 13

`operator!=`
 `Vector.h`, 18

`operator<`
 `Vector.h`, 18

`operator<=`
 `Vector.h`, 18

`operator>`
 `Vector.h`, 18

`operator>=`
 `Vector.h`, 18

`operator=`
 `Vector< T >`, 13

`operator==`
 `Vector.h`, 18

`operator[]`
 `Vector< T >`, 13, 14

`pointer`
 `Vector< T >`, 7

`pop_back`
 `Vector< T >`, 14

`push_back`
 `Vector< T >`, 14

`rbegin`
 `Vector< T >`, 14

`reference`
 `Vector< T >`, 7

`rend`
 `Vector< T >`, 14

`reserve`
 `Vector< T >`, 15

`resize`
 `Vector< T >`, 15

`reverse_iterator`
 `Vector< T >`, 8

- shrink_to_fit
 - Vector< T >, 15
- size
 - Vector< T >, 15
- size_type
 - Vector< T >, 8
- swap
 - Vector< T >, 15
 - Vector.h, 18
- value_type
 - Vector< T >, 8
- Vector
 - Vector< T >, 8, 9
- Vector< T >, 5
 - ~Vector, 9
 - allocator_type, 7
 - assign, 9
 - at, 9
 - back, 10
 - begin, 10
 - capacity, 10
 - cbegin, 10
 - cend, 10
 - clear, 10
 - const_iterator, 7
 - const_pointer, 7
 - const_reference, 7
 - const_reverse_iterator, 7
 - crbegin, 10
 - crend, 11
 - data, 11
 - difference_type, 7
 - emplace, 11
 - emplace_back, 11
 - empty, 11
 - end, 11
 - erase, 12
 - front, 12
 - get_allocator, 12
 - insert, 12, 13
 - iterator, 7
 - max_size, 13
 - operator=, 13
 - operator[], 13, 14
 - pointer, 7
 - pop_back, 14
 - push_back, 14
 - rbegin, 14
 - reference, 7
 - rend, 14
 - reserve, 15
 - resize, 15
 - reverse_iterator, 8
 - shrink_to_fit, 15
 - size, 15
 - size_type, 8
 - swap, 15
 - value_type, 8
 - Vector, 8, 9
 - Vector.h, 17
 - operator!=, 18
 - operator<, 18
 - operator<=, 18
 - operator>, 18
 - operator>=, 18
 - operator==, 18
 - swap, 18